Lesson 14: Raising the Stakes

Unity

VIDEO GAME CREATION

This week: later waves get genuinely harder - more enemies, moving faster - using the exact wave variable you've had since Lesson 11.

Open your own [YourName]Game project from Lesson 13 - don't start a new one.

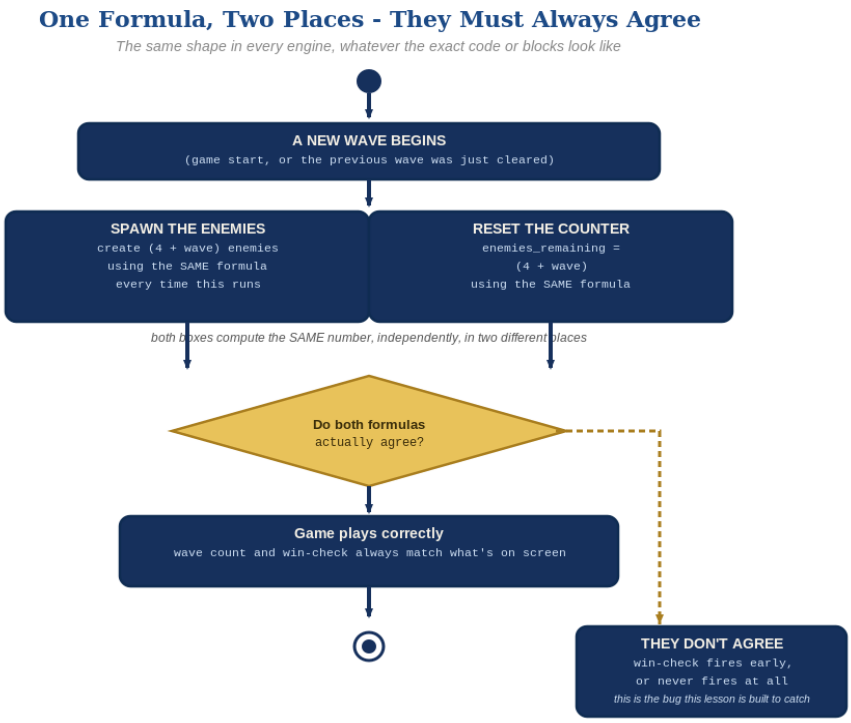
Tip: no new assets this week - difficulty scaling reuses your exact existing enemy sprites and spawn logic.

Setup

1. Open your [YourName]Game project from Lesson 13.
2. No new fields needed for Level 1 - Level 2 will use Enemy.cs's existing speed field (from Lesson 7).

Level 1 - Minimal (MVP)

Each wave spawns one more enemy than the last (5, 6, 7). Speed stays the same for now - that's Level 2.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

3. In EnemySpawner.cs's SpawnEnemies(), change the fixed loop count of 5 into (4 + GameManager.wave) (code below) - wave 1 spawns 5, wave 2 spawns 6, wave 3 spawns 7.
4. Wherever enemiesRemaining gets reset to 5 for a new wave, change that to 4 + wave too - these two places must always agree.
5. Press Play and play through a full round - wave 2 should visibly spawn one more enemy than wave 1, and wave 3 one more than that.

Highlighted lines are new this week; everything else is code you already have.

```
// EnemySpawner.cs:
public void SpawnEnemies() {
    int count = 4 + GameManager.wave;
    for (int enemyIndex = 0; enemyIndex < count; enemyIndex++) {
        GameObject newEnemy = Instantiate(enemyPrefab);
        // ...existing per-enemy positioning...
    }
    GameManager.enemiesRemaining = count;
}
```

Alternate Path: Visual Scripting

The same wave-dependent formulas work as Script Graph math nodes instead of C# expressions.

6. On your spawn-loop graph, replace the fixed loop count of 5 with an Add node combining 4 and the current Wave value.
7. On each spawned enemy, add a "Set Member" node targeting the enemy's own Enemy script (the same component Enemy.cs adds in the Script path) and its speed field, fed by an Add node combining 1.5 and (Wave - 1) times 0.5 - Set Member targets a real, type-checked field on the component (pick it from a list), so prefer it over the generic Variables > Set Variable node, which stores under a loose string key instead.

Level 2 - Complete (Alpha/Beta)

Enemy speed ALSO increases each wave, on top of the growing count. Restarting correctly returns everything to wave 1's original values.

- In SpawnEnemies(), right after instantiating each enemy, set its speed based on the wave: newEnemy.GetComponent<Enemy>().speed = 1.5f + (GameManager.wave - 1) * 0.5f.
- Confirm Enemy.cs's own speed field (from Lesson 7) is still what its Update() movement actually reads - no changes needed there.
- Restart mid-game and confirm both count and speed return correctly to wave 1's values.
- Play a full round and confirm wave 3 is noticeably busier and faster than wave 1, but still fair.

Level 3 - Refined - Choose ONE Stretch Feature

Pick exactly one of the five ideas below. These are intentionally open-ended - a good-faith attempt at the general idea is a complete Level 3 finish, not a fully polished feature.

- Destructible shields/barriers: a few simple blocks placed between the player and the enemies that disappear (or shrink) after absorbing a hit or two - reuses your existing collision-detection and hit-reaction patterns from Lessons 4 and 13, just on a new kind of object.
- A power-up: an item that occasionally drops or spawns, which the player collects to temporarily change how shooting works (faster fire rate, multiple bullets at once, or a brief shield) - reuses your existing timer/cooldown pattern from Lesson 3.
- A bonus enemy: a rare, faster-moving, or oddly-patterned enemy worth extra points, spawned occasionally alongside a normal wave - reuses this week's own speed/count scaling ideas, just for one special case instead of the whole wave.
- Formation-marching movement: instead of each enemy independently drifting down (this course's approach since Lesson 4), have the whole wave move together as a group and occasionally step sideways - a genuinely different movement pattern, inspired by classic arcade shooters.
- A title screen and/or game-over screen: a proper opening screen (press a key to start) and/or a dedicated game-over screen with a final score, instead of just text overlaid on the gameplay - reuses your existing game_active flag and UI tools from Lessons 9-10. Unity's UI Canvas system (already used for your HUD since Lesson 9) is a natural fit for a title/game-over screen.

If Something's Wrong

- Every wave still spawns exactly 5: confirm the loop actually uses (4 + GameManager.wave) and not a leftover literal 5.

- The win message fires early or never: confirm enemiesRemaining is reset using the SAME formula everywhere it's set.
- Speed doesn't change between waves: confirm each new enemy's speed field is being set INSIDE SpawnEnemies(), not left at the prefab's default forever.

Want More?

Student Resource List > Unity section, or learn.unity.com